

Flow-Field Pathfinding

Presentation Script & Technical Appendix

InternProj tower-defense prototype · speaker notes for a live, editor-free demo · July 25, 2026

This document is the script to talk from during the presentation. It assumes the Unity Editor will **not** be open — everything demoed here runs from a standalone build or a maximized Play-mode window, and every chart referenced lives in the companion **PerformanceReport.pdf**. A full technical appendix — architecture, the threading model, and exactly how each evaluation number was produced — follows the script starting on the page marked **Technical Appendix**, so you have precise answers ready for any follow-up question.

Before you start

- **Build a standalone player first.** All five scenes (SampleScene, Bench_Open, Bench_Maze, Bench_Choke, Bench_Stress) are already registered in File → Build Settings, so File → Build (or ask Claude to run it) produces a .exe that needs no editor. If you must present from Play mode instead, maximize the Game view (Ctrl/Cmd + drag the tab, or the maximize-on-play toggle) so no editor chrome is visible.
- **Have PerformanceReport.pdf open** on a second screen or as your slide deck — Segments 6–9 below point at specific pages and charts in it by number.
- **Know the controls** (also shown live in the game's own HUD, top-left): **V** cycles the vector-field view (Off → Arrows → Heatmap → Both) · **B** re-runs a benchmark scene's measurement · **WASD**/arrows pan the camera, scroll zooms, hold **right mouse** to free-look (**Q/E** to roll), **C** resets the camera · **Space** pauses, **1/2/3** set game speed · left click places the held piece, **Shift** swaps the held piece, **R** rotates it.
- **Rehearse the scene switch once.** Loading a different scene mid-build (Bench_Maze after the main game) takes a couple of seconds — know that beat so it doesn't feel like a stall live.

Numbers to have on the tip of your tongue

Field rebuild on the real game map: **~2.6 ms** — cheap enough to run on every wall edit.

Shared field vs. per-agent A* at 1,000 agents: **×55 cheaper** (2.8 ms vs. 153 ms).

500 live agents hold **~56–73 fps** across every scene tested, including a grid 6.9× larger.

Editing the map continuously under load roughly doubles frame time (17 ms → 44 ms avg) — the one real weak spot, and it's already got a named fix (coalesce rebuilds).

Suggested run of show (≈9–11 minutes)

Time	Segment	What's on screen
0:00–1:00	1 — Hook	Talking, no visuals yet
1:00–2:30	2 — Why a flow field	PerformanceReport.pdf, page 2
2:30–4:30	3 & 4 — Live game demo	Standalone build, SampleScene
4:30–6:00	5 — Benchmark scenes	Standalone build, Bench_Maze / Bench_Stress
6:00–9:00	6–9 — Walk the report	PerformanceReport.pdf, pages 4–6
9:00–10:00	10 — Wrap-up	PerformanceReport.pdf, page 6 (Recommendations)
10:00+	Q&A	Appendix in this document, as needed

The Script

Read SAY sections as a starting point, not a script to memorize verbatim — say it in your own words.

SEGMENT 1 The Hook	
ON SCREEN	Nothing yet — just you talking, or a title slide.
SAY	<i>This is a tower-defense prototype where the player builds Tetris-style wall pieces to slow down waves of enemies marching toward a core. Every enemy needs a path to that core, and that path has to update the instant the player drops a new wall — potentially with hundreds of enemies on screen at once. The question I want to answer today is: how does the pathfinding hold up, and how do I know?</i>

SEGMENT 2 Why a Flow Field (Not One Path Per Enemy)	
ON SCREEN	PerformanceReport.pdf, page 2, section 2 — "System under test".
SAY	<i>The naive approach is: every enemy runs its own pathfinding search to the core. That's wasteful, because every single enemy is searching for a path to the exact same destination. Instead, this project computes one shared "flow field": a single flood fill from the core outward that gives every tile on the map a direction to follow. Enemies don't search at all — they just read the arrow under their feet. The field gets rebuilt whenever the map changes, and walls aren't hard obstacles, they're expensive terrain with health, so a path always exists — enemies either detour around a wall or dig through it, whichever the field decides is cheaper.</i>

SEGMENT 3 Live Demo — The Game, With the Field Made Visible	
ON SCREEN	Standalone build, SampleScene, camera pulled back over the whole map.
SAY	<i>Here's the actual game running. Right now you can't see the pathfinding at all — that's normally invisible. Watch what happens when I press V. [press V twice to reach Arrows+Heatmap] Every tile is now colored by its cost to reach the core — green is cheap, red is expensive — and every tile has an arrow pointing the way an enemy standing there would go. This is the actual live data structure the AI reads from, rendered straight onto the ground in real time. It works in a full build with no editor attached, which is exactly why I can show it to you right now.</i>
DO	→ Press V until the caption bottom-left reads "ArrowsAndHeatmap" → Pan the camera (WASD / scroll) so the audience can see the whole grid

SEGMENT 4 Live Demo — Building a Wall and Watching It Reroute	
ON SCREEN	Same scene, mid-build phase (before wave 1 starts, or paused with Space).
SAY	<i>Now I'll place a wall. [click to place a piece] Watch the heatmap: the instant the piece lands, every arrow near it updates, because the field just rebuilt itself from scratch. That rebuild is not something you can see happen — it's fast enough to finish inside a single frame. If I start a wave, you'll see enemies path around this new wall on their own, or if it's cheap enough, walk straight into it and start digging.</i>
DO	→ Left-click to place the currently held piece → Optional: press Start (or wait) to spawn a wave and watch enemies react to the new wall

SEGMENT 5 Live Demo — The Benchmark Scenes	
ON SCREEN	Load Bench_Maze, then Bench_Stress, from the build's scene picker or by re-launching with that scene active.
SAY	<i>Alongside the main game there are four purpose-built scenes I made specifically to stress-test this system under controlled conditions. This is Bench_Maze — a hand-built serpentine corridor. Notice the heatmap bands: each corridor leg is a visibly more expensive shade of red than the last, because the cost keeps accumulating the further an enemy has to walk. This one is Bench_Stress — a 100-by-100 grid, roughly seven times the node count of the real game map, with a thousand randomly scattered walls. Both of these auto-run their own performance measurement the moment you hit Play, and you can press B to re-run it live if you want a real-time number on screen.</i>
DO	→ Press V to enable the heatmap in each new scene (the toggle resets per scene) → Press B to re-trigger a scenario's benchmark and narrate the console log as it reports

SEGMENT 6 The Report — Executive Summary	
ON SCREEN	PerformanceReport.pdf, page 1.
SAY	<i>Everything from here is measured, not estimated — every scene you just saw ran a real automated benchmark, and these are the actual numbers it produced. The headline is on this page: a rebuild on the real game map costs about 2.6 milliseconds, cheap enough to run on every single wall edit, and frame rate turns out to be limited by how many enemies are alive, not by the pathfinding itself.</i>
SEGMENT 7 The Report — Rebuild Scaling and the A* Comparison	
ON SCREEN	PerformanceReport.pdf, page 4, sections 4.1 and 4.2.
SAY	<i>This first chart shows rebuild cost as the grid grows and as more of it is walled off — it scales roughly linearly, and even at ten thousand nodes it's still only twenty-odd milliseconds. The second chart is the one I'd lead with in a Q&A: it directly compares this shared-field approach against giving every enemy its own A* search on identical hardware, identical grid, identical cost rules. At a thousand agents, the shared field is 55 times cheaper, and — this is the important part — that gap only gets wider as the crowd grows, because the field's cost doesn't depend on how many enemies are reading it.</i>
SEGMENT 8 The Report — Frame Rate vs. Crowd Size	
ON SCREEN	PerformanceReport.pdf, page 5, section 4.3.
SAY	<i>This is five scenes overlaid — the real game map, all four benchmark scenes — and they nearly sit on top of each other. That's the point: neither a more complex maze layout nor a grid seven times larger moves this curve. What actually costs frame time is the number of live agents, at roughly thirty-seven microseconds each, which is agent bookkeeping, not pathfinding. Five hundred agents hold sixty-plus frames a second everywhere; the game's actual waves peak around a hundred enemies, which runs at a hundred fifty-plus.</i>
SEGMENT 9 The Report — Editing the Map Under Load	
ON SCREEN	PerformanceReport.pdf, page 6, section 4.4, then section 5 (Recommendations).
SAY	<i>The one place this system shows real strain: if you keep editing the map continuously while five hundred agents are alive, average frame time roughly doubles, and the worst single frame hits eighty-five milliseconds. I dug into why — it's not the rebuild itself, it's that every single wall breach currently triggers its own immediate rebuild, so several breaches landing in the same frame each pay that cost separately. The fix is a one-line architectural change: coalesce rebuild requests into a dirty flag consumed once per frame instead of firing immediately. That's the top recommendation in the report, and it's the honest, unglamorous kind of finding I wanted this evaluation to be able to surface.</i>

SEGMENT 10 Wrap-Up

ON SCREEN PerformanceReport.pdf, page 6 — Recommendations list.

SAY *So: the architecture choice — one shared field instead of per-agent search — is validated by the numbers, not just intuition. It scales to far more agents and a far bigger map than this game actually needs, the one weak spot has a specific known fix, and everything you just watched happen live was rendered by the same visualization tool that made this evaluation possible to build and check in the first place. Happy to take questions.*

Anticipated Questions

Question	Answer
"Is this actually Dijkstra's algorithm?"	Close, but precisely: it's SPFA (a queue-based Bellman-Ford variant) — a node can be re-added to the queue if a cheaper route to it is found later, rather than using a priority queue. On this grid's cost structure it behaves like Dijkstra in practice; the A* used for the comparison chart is a real binary-heap implementation, which is why that comparison is apples-to-apples on cost model, not on algorithm family.
"Does this use multithreading / the Job System?"	No — zero multithreading anywhere in the project, confirmed by search. Full explanation, including why an earlier version of this code did use background threads and had them deliberately removed, is in the Technical Appendix, section B.
"How do you know these numbers are real and not made up?"	Every chart in the report is generated directly from JSON files the benchmark code itself writes at runtime — nothing is hand-typed. Appendix section E walks through exactly how each statistic is computed, line by line.
"What happens with even more agents, like 5,000?"	Not tested here — the tiers stop at 1,000 because that's an order of magnitude past the game's real wave sizes (~100). The fps-vs-agents curve is linear in the range measured, so a rough extrapolation exists, but it would need its own benchmark run to state as a real number.
"Why not use Unity's built-in NavMesh?"	NavMesh is built for per-agent goals on largely static geometry; this game needs many agents converging on one shared destination while the walkable surface changes every few seconds as the player builds — a flow field amortizes exactly that shared-destination, frequently-changing-map case in a way NavMesh isn't designed for.

Technical Appendix

Everything you need to know about how this project works under the hood, and exactly how every evaluation number in PerformanceReport.pdf was produced.

A. Architecture at a Glance

Script	Role
Node.cs	Plain C# class (not a struct) — one grid cell: walkability, terrain cost, wall health, and the field's own bestCost / bestDirection / nextNode.
GridManager.cs	Builds the Node[,] grid from world bounds at Awake(); world↔grid coordinate conversion; 8-neighbor lookup with a corner-cutting rule.
FlowFieldManager.cs	Owns GenerateFlowField() — the flood fill described in section C. Bumps a FieldVersion counter after every rebuild.
FlowAgent.cs	Per-enemy behaviour: follows nextNode each frame, chews a wall's health down while standing on it, dies on timeout or reaching the core.
PlayerBuilder.cs / BlockManager.cs / BlockShape.cs	Piece queue (7-bag randomizer), placement legality, and the synchronous rebuild triggered by every placement.
FlowFieldVisualizer.cs	New this evaluation — renders the field as ground geometry. Detailed in section D.
ScenarioBenchmark.cs / BenchmarkRunner.cs	The measurement harnesses. Detailed in section E.

B. Threading & Concurrency Model

Short answer

This project uses **no multithreading anywhere**. There is no `Unity Job System`, no `Burst compiler`, no `C# System.Threading.Thread`, no `Task/async-await`, and no `Parallel`. For in any script under `Assets/Scripts` (verified by a full-project text search). Grid construction, the flow-field flood fill, every agent's movement and digging, and all benchmark sampling run synchronously on Unity's single main thread, one operation after another, once per frame.

It's worth being precise about **coroutines**, since they're easy to mistake for concurrency: WaveSpawner's wave/intermission timers, CanvasDashboard's UI pop animations, and the benchmark scripts' sampling loops are all Unity coroutines (`IEnumerator + yield return`). A coroutine is **cooperative multitasking on the main thread** — Unity pauses the method at a `yield` and resumes it on a later frame, but it never executes concurrently with anything else. A coroutine and `Update()` never run at the same instant; a coroutine and `Update()` also never race over shared data the way two real threads could. There is exactly one thread doing exactly one thing at a time throughout this whole project.

This project used to use background threads — and removed them

The comments in the current code are explicit about this history. An earlier version of the pathfinding ran the flood fill on a background thread, split the map into chunks, and needed an `isCalculating` lock plus a `validate/revert` step for whenever a background result arrived after the map had already changed again:

```
// One synchronous weighted flood fill (Dijkstra-style relaxation), run on the
// main thread whenever the map changes. On a 25x25 grid this costs microseconds
// per call, which is why the old chunk system, background threads, the
// isCalculating lock, ValidatePath, sinkhole detection and the global fallback
// could all be deleted: there is nothing left to go stale between chunks, and
// no background thread for agents or previews to race against.
— FlowFieldManager.cs, class header
```

And directly in the wall-placement code:

```
// Placement is fully synchronous now. Because walls are expensive terrain
// rather than absolute blockers, NO placement can ever disconnect the map -
// so there is nothing to validate asynchronously, nothing to revert, and no
// background thread to race against.
— PlayerBuilder.cs, HandlePlacement()
```

The underlying design change that made this possible: walls became *expensive terrain* (a high but finite cost) instead of hard obstacles. That single change means a path to the goal always mathematically exists, so there's nothing to validate, nothing to roll back if a background calculation turns out stale, and therefore no reason to pay the complexity cost of a lock or a background thread at all. The 2.6 ms rebuild time measured on the real game map (section E.1) is the empirical proof that the synchronous version is fast enough that this simplification was the right call, not a premature one.

When would multithreading actually start to matter here?

Section 4.1 of the report shows rebuild cost growing to ~19–40 ms at a 100×100 grid (10,000 nodes) — that's the point where a synchronous rebuild starts to compete visibly with a 60 fps frame budget, and where moving the flood fill off the main thread (or amortizing it across several frames) would start to pay for itself. Note this isn't a drop-in change: Node is a plain C# reference-type class with object pointers (nextNode, visualObject), which the Job System/Burst combination can't touch directly — adopting it would mean restructuring the grid into blittable NativeArrays first. A plain System.Threading.Thread or Task.Run off the main thread would be the lower-effort option, at the cost of reintroducing exactly the staleness/locking problem the code comments above describe removing.

C. The Pathfinding Algorithm

`FlowFieldManager.GenerateFlowField()` runs a single-source shortest-path fill outward from the goal (the player's core), storing the result at every node instead of tracing one path per query:

- 1 Every node's cost is reset to infinity and its `nextNode` pointer cleared, so no stale pointer can survive a recalculation.
- 2 The goal node is seeded at cost 0 and pushed onto a FIFO queue.
- 3 While the queue isn't empty: pop a node, examine its up-to-8 neighbours, and for each one compute *step cost* = (10 for a cardinal move, 14 for a diagonal — an integer approximation of $10 \times \sqrt{2}$) $\times$ the neighbour's `terrainCost`. If that beats the neighbour's current best cost, update it, point its `nextNode` back at the current node, and re-enqueue it.
- 4 A node can be enqueued more than once if a cheaper route to it is found later — this is what makes it **SPFA** (Shortest Path Faster Algorithm, a queue-based Bellman-Ford variant) rather than textbook Dijkstra, which would use a priority queue and settle each node exactly once. On this grid's cost structure (small integer weights, no negative edges) it produces identical results to Dijkstra; it's simply implemented with a plain queue instead of a binary heap.
- 5 Every reachable node ends up with a cost-to-goal and a single "best neighbour" pointer. Any agent, anywhere on the map, gets its next step in $O(1)$ by reading its current node's `nextNode` — no search happens at query time at all.

Two extra rules shape the result: a **corner-cutting rule** in `GridManager.GetNeighbors()` forbids a diagonal move when either of the two flanking cardinal tiles is unwalkable or has a standing wall — an agent may walk straight into a wall (that's how digging starts) but may never slip diagonally between two solid tiles. And **walls are terrain, not obstacles**: a placed wall raises its tile's `terrainCost` (default dig cost 15, meaning "as bad as a 15-tile detour") rather than removing it from the graph, and separately tracks a `wallHealth` that an agent standing on it depletes over time. When health hits zero the tile resets to open ground and `GenerateFlowField()` is called again immediately so every agent reroutes through the fresh gap.

D. The Runtime Vector-Field Visualizer

`FlowFieldVisualizer.cs` (new this evaluation) renders the field as two separate procedural meshes parented under the same `GameObject` as the flow field manager:

- **Heatmap layer** — one quad per walkable node, vertex-colored on a green→red ramp normalized to the highest reachable cost on the map that frame (orange = standing wall, dark grey = static blocker, purple = unreachable pocket).
- **Arrow layer** — a small triangulated arrow per node pointing along its `bestDirection`, plus a diamond marker on the goal tile; skipped on wall tiles since the wall geometry would hide it anyway.
- Both layers use an unlit, vertex-colored material (`Sprites/Default` shader, falling back from a `Resources/FlowFieldViz` asset) so the whole overlay is one draw call per layer regardless of grid size, and use 32-bit mesh indices so grids above 65,536 vertices (e.g. the 100×100 stress scene) don't overflow Unity's default 16-bit index buffer.
- Geometry is only rebuilt when `FlowFieldManager.FieldVersion` changes or the view mode changes — the counter is bumped once at the end of every `GenerateFlowField()` call — so on a static field the visualizer's steady-state per-frame cost is a single integer comparison, not a re-render.
- The mode cycles Off → Arrows → Heatmap → Arrows+Heatmap on the **V** key (`Keyboard.current` from Unity's new Input System — this project does not use the legacy `Input` class anywhere), with a small always-on-top `OnGUI()` label showing the current mode.

E. How Every Evaluation Statistic Was Produced

Nothing in PerformanceReport.pdf is estimated or hand-entered. Every scene writes a JSON file straight to the project root at the end of its run, and the report's charts are generated directly from those files (Tools/make_report.py). This section explains exactly what each number measures.

E.1 — Rebuild time (mean / min / max, ms)

A `System.Diagnostics.Stopwatch` is started and stopped around a single call to `FlowFieldManager.GenerateFlowField()`. Three warm-up calls run first and are discarded (letting the JIT and CPU caches settle), then the call is timed individually for 30 iterations (BenchmarkRunner's grid-size × wall-density sweep) or 50 iterations (each ScenarioBenchmark scene). Mean, min and max are computed from those raw per-call millisecond samples — this is wall-clock cost of the exact algorithm in section C, not a theoretical estimate.

E.2 — Path metrics (tiles, cost)

After a fresh rebuild, the spawn point's node is located, then the code walks the `nextNode` chain — the field's own back-pointers — from spawn to goal, counting hops for "tiles" and reading the spawn node's `bestCost` for "cost". Cost is stored internally ×10 (so the 14/10 diagonal ratio can be an integer) and divided by 10 for display, so it reads in intuitive "tile-equivalent" units.

E.3 — Frame time / fps per agent tier

- 1 A pooled set of agent `GameObjects` is grown to the target tier size (100/250/500/1,000), each initialized with an effectively infinite lifetime (999,999 s) so none expire mid-measurement and skew the sample.
- 2 A 1-second, unsampled warm-up lets the freshly spawned agents spread out and any spawn-frame hitch settle before any timing starts.
- 3 Then, for a 4-second sampling window, every single frame: any agent that has reached the goal is immediately respawned at the spawn point (keeping the live count exactly constant across the whole window), and `Time.unscaledDeltaTime` for that frame is appended to a list.
- 4 From that list: **avg** = mean of all sampled frame times; **fps** = 1 / avg; the list is sorted and **p95** is the value at the 95th-percentile rank; **worst** is simply the single largest frame time observed in the whole window (not an average of bad frames — the literal worst individual frame).
- 5 Unscaled time is used deliberately so the sample is unaffected by the game's own x1/x2/x3 time-scale control, and `vSync` is forced off with the frame rate uncapped (`QualitySettings.vSyncCount = 0`; `Application.targetFrameRate = -1`) at the start of every run, so the numbers reflect the engine's real per-frame cost rather than a monitor's refresh ceiling.

E.4 — Shared field vs. per-agent A*

Both approaches run on the identical grid (38×38, 10% wall density, the same 10/14×terrainCost step-cost model) so the comparison isolates the algorithmic choice, not the map. "Field rebuild ms" is one `Stopwatch`-timed call as in E.1. "Per-agent A*" is a separate, hand-written implementation — a binary min-heap (array-backed, explicit sift-up/sift-down, not a library structure) with an octile-distance heuristic that is admissible because the cheapest possible tile costs `terrainCost = 1`. It runs once per random walkable start tile toward the same goal (one warm-up search discarded first), and the reported number is one `Stopwatch` span covering the whole batch of N searches back to back — so it is genuinely "what it would cost to path N enemies individually," not a per-search figure multiplied out.

E.5 — Dynamic-edit stability test

With 500 agents held alive exactly as in E.3, a coroutine toggles a 2×2 block of wall tiles on or off every 0.5 seconds for 8 seconds (16 edits total), calling `GenerateFlowField()` synchronously after every single edit. This is a deliberately worse case than real play — the actual game only rebuilds on a genuine player

placement or a wall breach, not a fixed timer — chosen specifically to surface a worst-case number. Frame times are sampled with the identical FrameSampler used in E.3 throughout the 8-second window and compared against the static-map 500-agent tier from the same run.

E.6 — Environment metadata

CPU/GPU/RAM/OS strings and core count are read from Unity's SystemInfo API and Application.unityVersion at the start of every run and written verbatim into that run's JSON file, so each result file is self-describing and traceable to the exact machine it ran on.

F. Methodology Caveats

- **Single machine, single run per condition.** These are point measurements, not averages across repeated trials — treat them as one representative sample rather than statistically bulletproof figures. Re-running the same scene (press B) typically reproduces results within a few percent on this hardware.
- **Editor Play Mode, not a standalone build.** All numbers in this evaluation were captured with the Unity Editor running (vSync off, uncapped frame rate) — editor-only overhead (profiler hooks, debug scaffolding) makes these a conservative floor. An actual standalone build should perform at least as well.
- **Layout determinism varies by test.** Bench_Maze and Bench_Choke build the same wall layout every run by construction; Bench_Stress seeds its random scatter (Random.InitState(12345)) so it's reproducible too. BenchmarkRunner's grid-size × density sweep (used for the report's section 4.1 chart) places walls with live, unseeded Random.Range calls, so that specific chart can show minor run-to-run variance in exactly which tiles are walled, though not in the density.
- **Uncapped frame rate is intentional but unrealistic for players.** Turning vSync off exposes true per-frame cost, which is what a performance evaluation needs — but most players will actually run with vSync on, where frame pacing looks smoother than these raw numbers suggest.
- **Benchmark scenes are static snapshots.** Bench_Maze/Choke/Stress don't grow their walls over time the way a real player's match does — they measure the cost of a given map complexity, not the cost of a map that keeps escalating for 20+ waves. E.5 is the one test that specifically adds ongoing map churn.

G. Glossary

Term	Meaning
Flow field	A grid where every tile stores a direction toward one shared destination, computed once and read by every agent.
SPFA	Shortest Path Faster Algorithm — a FIFO-queue-based relaxation (Bellman-Ford family); this project's fill algorithm.
Terrain cost	The cost to enter a tile. 1 = open ground; a placed wall raises it (default 15) to represent "as bad as a 15-tile detour."
Dig / wall health	Seconds of chewing (by one agent; several agents chew a shared tile in parallel) before a wall tile breaks back to open ground.
Corner-cutting rule	Diagonal moves are forbidden when either flanking cardinal tile is solid, so agents can't clip through wall corners.
FieldVersion	A counter bumped once per completed rebuild, used by the visualizer to know when to regenerate its mesh.
p95 / worst	p95 = the frame time only the worst 5% of sampled frames exceeded. Worst = the single slowest frame in the sample.
Coroutine	Cooperative multitasking on Unity's main thread (pause/resume between frames) — not a thread, no real concurrency.